

Trabajo Práctico N° 4: **Tiempos de Ejecución.**

Ejercicio 1.

Debido a un error en la actualización de sus sistemas, el banco AyED perdió la información del estado de todas sus cuentas. Afortunadamente, logran recuperar un backup del día anterior y, utilizando las transacciones registradas en las últimas 24hrs, podrán reconstruir los saldos. Hay poco tiempo que perder, el sistema bancario debe volver a operar lo antes posible.

Las transacciones se encuentran agrupadas en consultas, una consulta cuenta con un valor y un rango de cuentas consecutivas a las que hay que aplicar este cambio, por ejemplo la consulta (333..688= 120) implica sumar \$120 a todas las cuentas entre la número 333 y la número 688 (inclusive). Entonces, la recuperación de los datos consiste en aplicar todas las consultas sobre el estado de las cuentas recuperadas en el backup del día anterior.

El equipo de desarrollo se pone manos a la obra y llega a una solución rápidamente (Algoritmo procesarMovimientos). Toman cada consulta y recorren el rango de cuentas aplicando el valor correspondiente, como muestra el siguiente algoritmo.

Consultas.comenzar()

```
While(!consultas.fin()){  
    Consulta = consultas.proximo();  
    for(i = consulta.desde; i < consulta.hasta; i++){  
        cuenta[i] = cuenta[i] + consulta.valor;  
    }  
}
```

Escriben la solución en pocos minutos y ponen en marcha el proceso de recuperación. Enseguida se dan cuenta que el proceso va a tardar muchas horas en finalizar, son muchas cuentas y muchos movimientos, la solución aunque simple es ineficiente. Luego de discutir varias ideas llegan a una solución (Algoritmo procesarMovimientosOptimizado) que logra procesar toda la información en pocos segundos. Ambos algoritmos se encuentran en el archivo Ejercicio 1 - rsq_tn_ayed.zip del material adicional.

(a) Para que se pueda experimentar el tiempo que demora cada uno de los dos algoritmos en forma empírica, se debe ejecutar cada uno de ellos, con distintas cantidades de elementos y completar la tabla. Luego, hacer la gráfica para comparar los tiempos de ambos algoritmos. Tener en cuenta que el algoritmo posee dos constantes CANTIDAD_CUENTAS y CANTIDAD_CONSULTAS, sin embargo, por simplicidad, ambas toman el mismo valor. Sólo se necesita modificar CANTIDAD_CUENTAS.

Nº Cuentas (y consultas)	procesarMovimientos	procesarMovimientosOptimizado
1.000	0,034	0,001
10.000	0,036	0,003
25.000	0,034	0,006
50.000	0,033	0,011
100.000	0,046	0,019

Ver paquete “tp4.ejercicio1” en Java.

(b) *¿Por qué procesarMovimientos es tan ineficiente? Tener en cuenta que pueden existir millones de movimientos diarios que abarquen gran parte de las cuentas bancarias.*

El método tiene una complejidad temporal de $O(N \cdot M)$, donde N es la cantidad de consultas y M es el promedio del tamaño de los rangos de cuentas afectadas por cada consulta. En el caso que haya muchas consultas o los rangos sean grandes, este algoritmo sería muy ineficiente.

(c) *¿En qué se diferencia procesarMovimientosOptimizado? Observar las operaciones que se realizan para cada consulta. Aunque los dos algoritmos se encuentran explicados en los comentarios, no es necesario entender su funcionamiento para contestar las preguntas.*

En lo que se diferencia *procesarMovimientosOptimizado* es en que hace un único recorrido tanto en el arreglo de consultas como el de cuentas. Esto es así gracias a la ayuda de un arreglo auxiliar en donde se registran los cambios que deben aplicarse a las cuentas sumando los valores correspondientes (en el anterior algoritmo, los cambios se realizaban, directamente, dentro del bucle anidado).

Ejercicio 2.

La clase `BuscadorEnArrayOrdenado` del material adicional (Ejercicio 2 - Tiempo.zip) resuelve el problema de buscar un elemento dentro de un array ordenado. El mismo problema lo resuelve de dos maneras diferentes: búsqueda lineal y búsqueda dicotómica.

Se define la variable `cantidadElementos`, la cual se va modificando para determinar una escala (por ejemplo, de a 100.000 o 1.000.000, dependiendo de la capacidad de cada equipo). Realizar una tabla con el tiempo que tardan en ejecutarse ambos algoritmos, para los distintos valores de la variable.

N	Lineal	Dicotómica
100.000	0,002	0,000
200.000	0,005	0,000
300.000	0,006	0,000
400.000	0,007	0,000
500.000	0,008	0,000
600.000	0,011	0,000

Ver paquete “tp4.ejercicio2” en Java.

Ejercicio 3.

En la documentación de la clase `ArrayList` que se encuentra en el siguiente link <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>, se encuentran las siguientes afirmaciones:

- “The `size`, `isEmpty`, `get`, `set`, `iterator`, and `listIterator` operations run in constant time.”
- “All of the other operations run in linear time (roughly speaking).”

Explicar por qué se cree que algunas operaciones se ejecutan en tiempo constante y otras en tiempo lineal.

Las operaciones como `size`, `isEmpty`, `get`, `set`, `Iterator` y `listIterator` se ejecutan en tiempo constante porque acceden a posiciones de memoria o propiedades simples. En cambio, el resto de las operaciones se ejecutan en tiempo lineal porque requieren mover o procesar varios elementos.

Ejercicio 4.

Determinar si las siguientes sentencias son verdaderas o falsas, justificando la respuesta utilizando notación Big-Oh.

(a) 3^n es de $O(2^n)$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

$$3^n \leq c 2^n$$

$$\frac{3^n}{2^n} \leq c$$

$$\left(\frac{3}{2}\right)^n \leq c.$$

La sentencia es FALSA, ya que no existen $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

(b) $n + \log_2(n)$ es de $O(n)$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

$$n + \log_2(n) \leq cn$$

$$\log_2(n) \leq cn - n$$

$$\log_2(n) \leq n(c - 1)$$

$$\log_2(n) \leq n(2 - 1)$$

$$\log_2(n) \leq n * 1$$

$$\log_2(n) \leq n$$

$$n \geq 1.$$

La sentencia es VERDADERA, ya que existen $c = 2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

(c) $n^{\frac{1}{2}} + 10^{20}$ es de $O(n^{\frac{1}{2}})$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

$$n^{\frac{1}{2}} + 10^{20} \leq c n^{\frac{1}{2}}$$

$$10^{20} \leq c n^{\frac{1}{2}} - n^{\frac{1}{2}}$$

$$10^{20} \leq n^{\frac{1}{2}}(c - 1)$$

$$10^{20} \leq n^{\frac{1}{2}}(2 - 1)$$

$$10^{20} \leq n^{\frac{1}{2}} * 1$$

$$10^{20} \leq n^{\frac{1}{2}}$$

$$n \geq (10^{20})^2$$

$$n \geq 10^{40}.$$

La sentencia es VERDADERA, ya que existen $c=2$ y $n_0=10^{40}$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

(d) $\begin{cases} 3n+17, n < 100 \\ 317, n \geq 100 \end{cases}$ tiene orden lineal.

La sentencia es VERDADERA, ya que el orden de mayor grado es $O(n)$.

(e) Mostrar que $p(n) = 3n^5 + 8n^4 + 2n + 1$ es $O(n^5)$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $p(n) \leq c g(n)$ para todo $n \geq n_0$?

$$3n^5 + 8n^4 + 2n + 1 \leq cn^5.$$

Primer término:

$$3n^5 \leq cn^5$$

$$3n^5 \leq 3n^5.$$

$$c_1 = 3; n_1 = 1.$$

Segundo término:

$$8n^4 \leq cn^5$$

$$8n^4 \leq 8n^5.$$

$$c_2 = 8; n_2 = 1.$$

Tercer término:

$$2n \leq cn^5$$

$$2n \leq 2n^5.$$

$$c_3 = 2; n_3 = 1.$$

Cuarto término:

$$1 \leq cn^5$$

$$1 \leq 1n^5$$

$$1 \leq n^5.$$

$$c_4 = 1; n_4 = 1.$$

Entonces:

$$3n^5 + 8n^4 + 2n + 1 \leq c_1 n^5 + c_2 n^5 + c_3 n^5 + c_4 n^5$$

$$3n^5 + 8n^4 + 2n + 1 \leq (c_1 + c_2 + c_3 + c_4) n^5$$

$$3n^5 + 8n^4 + 2n + 1 \leq (3 + 8 + 2 + 1) n^5$$

$$3n^5 + 8n^4 + 2n + 1 \leq 14n^5, \text{ con } c = 14 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $p(n)$ es $O(n^5)$, ya que existen constantes $c = 14$ y $n_0 = 1$ tales que $p(n) \leq c g(n)$ para todo $n \geq n_0$.

(f) Si $p(n)$ es un polinomio de grado k , entonces, $p(n)$ es $O(n^k)$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $p(n) \leq c g(n)$ para todo $n \geq n_0$?

La sentencia es VERDADERA, ya que el término de mayor grado k domina a medida que $n \rightarrow \infty$, siendo los términos de orden inferior (n^{k-1} , n^{k-2} , ...) despreciables frente a n^k .

Ejercicio 5.

Se necesita generar una permutación random de los n primeros números enteros. Por ejemplo, $[4, 3, 1, 0, 2]$ es una permutación legal, pero $[0, 4, 1, 2, 4]$ no lo es, porque un número está duplicado (el 4) y otro no está (el 3). Se presentan tres algoritmos para solucionar este problema. Se asume la existencia de un generador de números random, $\text{ran_int}(i, j)$, el cual genera, en tiempo constante, enteros entre i y j inclusive con igual probabilidad (esto significa que puede retornar el mismo valor más de una vez). También se supone el mensaje $\text{swap}()$ que intercambia dos datos entre sí.

```
public class EjercicioPermutaciones {
    private static Random rand = new Random();

    public static int[] randomUno(int n) {
        int i, x = 0, k;
        int[] a = new int[n];
        for (i = 0; i < n; i++) {
            boolean seguirBuscando = true;
            while (seguirBuscando) {
                x = ran_int(0, n - 1);
                seguirBuscando = false;
                for (k = 0; k < i && !seguirBuscando; k++)
                    if (x == a[k])
                        seguirBuscando = true;
            }
            a[i] = x;
        }
        return a;
    }

    public static int[] randomDos(int n) {
        int i, x;
        int[] a = new int[n];
        boolean[] used = new boolean[n];
        for (i = 0; i < n; i++) used[i] = false;
        for (i = 0; i < n; i++) {
            x = ran_int(0, n - 1);
            while (used[x]) x = ran_int(0, n - 1);
            a[i] = x;
            used[x] = true;
        }
        return a;
    }
}
```

```

public static int[] randomTres(int n) {
    int i;
    int[] a = new int[n];
    for (i = 0; i < n; i++) a[i] = i;
    for (i = 1; i < n; i++) swap(a, i, ran_int(0, i - 1));
    return a;
}

private static void swap(int[] a, int i, int j) {
    int aux;
    aux = a[i]; a[i] = a[j]; a[j] = aux;
}

/** Genera en tiempo constante, enteros entre i y j con igual probabilidad.
 */
private static int ran_int(int a, int b) {
    if (b < a || a < 0 || b < 0) throw new IllegalArgumentException("Parametros
    invalidos");
    return a + (rand.nextInt(b - a + 1));
}

public static void main(String[] args) {
    System.out.println(Arrays.toString(randomUno(1000)));
    System.out.println(Arrays.toString(randomDos(1000)));
    System.out.println(Arrays.toString(randomTres(1000)));
}

```

(a) Analizar si todos los algoritmos terminan o alguno puede quedar en loop infinito.

Los primeros dos algoritmos podrían quedar en un *loop* infinito sólo si el generador de números aleatorios retorna siempre el mismo número. Ambos algoritmos están diseñados para que terminen una vez hallada una permutación legal y, en caso contrario, sigan iterando hasta hallarla.

En el último de los algoritmos, en caso de que el generador de números aleatorios retorne siempre el mismo número, la posición intercambiada sería la misma, pero no generaría un *loop* infinito.

Ver paquete “tp4.ejercicio5” en Java.

(b) Describir, con palabras, la cantidad de operaciones que realizan.

- *randomUno*: Para cada valor, compara con los anteriores hasta encontrar uno no repetido (con posibles repeticiones).
- *randomDos*: Usa *used[]* para evitar repeticiones y sólo realiza reintentos si hay colisiones.
- *randomTres*: Llena el arreglo y hace un intercambio aleatorio por posición.

Ejercicio 6.

(a) Se supone que se tiene un algoritmo de $O(\log_2^2 n)$ y se dispone de 1 hora de uso de CPU. En esa hora, la CPU puede ejecutar el algoritmo con una entrada de tamaño $n=1.024$ como máximo. ¿Cuál sería el mayor tamaño de entrada que podría ejecutar el algoritmo si se dispone de 4 horas de CPU?

En 1 hora, $n=1.024$:

$$(\log_2 n)^2 = (\log_2 1024)^2$$

$$(\log_2 n)^2 = 10^2$$

$$(\log_2 n)^2 = 100.$$

En 4 horas:

$$(\log_2 n)^2 = 4 * 100$$

$$(\log_2 n)^2 = 400$$

$$\log_2 n = \sqrt{400}$$

$$\log_2 n = 20$$

$$n = 2^{20}$$

$$n = 1048576.$$

Por lo tanto, el mayor tamaño de entrada que podría ejecutar el algoritmo si se dispone de 4 horas de CPU es 1.048.576.

(b) Considerando que un algoritmo requiere $T(n)$ operaciones para resolver un problema y la computadora procesa 10.000 operaciones por segundo. Si $T(n) = n^2$, determinar el tiempo en segundos requerido por el algoritmo para resolver un problema de tamaño $n=2.000$.

$$T(n) = n^2.$$

$$T(2000) = 2000^2$$

$$T(2000) = 4000000.$$

10.000 operaciones por segundo.

$$\text{Tiempo (en segundos)} = \frac{4000000}{10000}$$

$$\text{Tiempo (en segundos)} = 400.$$

Por lo tanto, el tiempo en segundos requerido por el algoritmo para resolver un problema de tamaño $n=2.000$ es 400.

Ejercicio 7.

Para cada uno de los siguientes fragmentos de código, calcular, intuitivamente, el orden del tiempo de ejecución.

<pre>for(int i = 0; i < n; i++) sum++;</pre>	<pre>for(int i = 0; i < n; i+=2) sum++;</pre>
<pre>for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) sum++;</pre>	<pre>for(int i = 0; i < n + 100; ++i) { for(int j = 0; j < i * n; ++j){ sum = sum + j; } for(int k = 0; k < n + n + n; ++k){ c[k] = c[k] + sum; } }</pre>
<pre>for(int i = 0; i < n; i++) for(int j = 0; j < n; j++) sum++; for(int i = 0; i < n; i++) sum++;</pre>	<pre>int i,j; int x = 1; for (i = 0; i <= n²; i=i+2) for (j = n; j >= 1; j-= n/4) x++;</pre>

(a)

Iteraciones del *for*:

i= 0.
i= 1.
i= 2.
...
i= k - 1.

k - 1= n - 1
k= n - 1 + 1
k= n.

Entonces:

$T(n) = \sum_{i=1}^n cte$
 $T(n) = n \text{ cte.}$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n)$.

(b)

Iteraciones del *for*:

i= 0.
i= 2.

$i = 4.$

...

$i = 2(k - 1).$

$2(k - 1) = n - 1$

$k - 1 = \frac{n-1}{2}$

$k = \frac{n-1}{2} + 1$

$k = \frac{n+1}{2}.$

Entonces:

$T(n) = \sum_{i=1}^{\frac{n+1}{2}} cte$

$T(n) = \frac{n+1}{2} cte.$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n)$.

(c)

Iteraciones del primer *for*:

$i = 0.$

$i = 1.$

$i = 2.$

...

$i = k - 1.$

$k - 1 = n - 1$

$k = n - 1 + 1$

$k = n.$

Iteraciones del segundo *for*:

$j = 0.$

$j = 1.$

$j = 2.$

...

$j = k - 1.$

$k - 1 = n - 1$

$k = n - 1 + 1$

$k = n.$

Entonces:

$T(n) = \sum_{i=1}^n \sum_{j=1}^n cte$

$$T(n) = \sum_{i=1}^n n * cte$$

$$T(n) = nn \text{ cte}$$

$$T(n) = n^2 \text{ cte.}$$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n^2)$.

(d)

Iteraciones del primer *for*:

$i = 0.$

$i = 1.$

$i = 2.$

...

$i = k - 1.$

$k - 1 = n + 100 - 1$

$k = n + 100 - 1 + 1$

$k = n + 100.$

Iteraciones del segundo *for*:

$j = 0.$

$j = 1.$

$j = 2.$

...

$j = k - 1.$

$k - 1 = in - 1$

$k = in - 1 + 1$

$k = in.$

Iteraciones del tercer *for*:

$k = 0.$

$k = 1.$

$k = 2.$

...

$k = k' - 1.$

$k - 1 = 3n - 1$

$k = 3n - 1 + 1$

$k = 3n.$

Entonces:

$$T(n) = \sum_{i=1}^{n+100} (\sum_{j=1}^{in} cte_1 + \sum_{k=1}^{3n} cte_2)$$

$$T(n) = \sum_{i=1}^{n+100} (in * cte_1 + 3n * cte_2)$$

$$\begin{aligned}
T(n) &= \sum_{i=1}^{n+100} in * cte_1 + \sum_{i=1}^{n+100} 3n * cte_2 \\
T(n) &= n cte_1 \sum_{i=1}^{n+100} i + (n + 100) 3n cte_2 \\
T(n) &= n cte_1 \frac{(n+100)(n+100+1)}{2} + (n + 100) 3n cte_2 \\
T(n) &= n cte_1 \frac{(n+100)(n+101)}{2} + (n + 100) 3n cte_2 \\
T(n) &= \frac{(n+100)(n+101)n}{2} cte_1 + (3n^2 + 300n) cte_2 \\
T(n) &= \frac{n^3 + 101n^2 + 100n^2 + 10100n}{2} cte_1 + (3n^2 + 300n) cte_2 \\
T(n) &= \frac{n^3 + 201n^2 + 10100n}{2} cte_1 + (3n^2 + 300n) cte_2 \\
T(n) &= \left(\frac{1}{2}n^3 + \frac{201}{2}n^2 + 5050n\right) cte_1 + (3n^2 + 300n) cte_2.
\end{aligned}$$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n^3)$.

(e)

Iteraciones del primer *for*:

$i = 0.$
 $i = 1.$
 $i = 2.$
 $\dots$
 $i = k - 1.$

$k - 1 = n - 1$
 $k = n - 1 + 1$
 $k = n.$

Iteraciones del segundo *for*:

$j = 0.$
 $j = 1.$
 $j = 2.$
 $\dots$
 $j = k - 1.$

$k - 1 = n - 1$
 $k = n - 1 + 1$
 $k = n.$

Iteraciones del tercer *for*:

$i = 0.$
 $i = 1.$
 $i = 2.$
 $\dots$
 $i = k - 1.$

$$k - 1 = n - 1$$

$$k = n - 1 + 1$$

$$k = n.$$

Entonces:

$$T(n) = \sum_{i=1}^n \sum_{j=1}^n cte_1 + \sum_{i=1}^n cte_2$$

$$T(n) = \sum_{i=1}^n n * cte_1 + n cte_2$$

$$T(n) = nn cte_1 + n cte_2$$

$$T(n) = n^2 cte_1 + n cte_2.$$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n^2)$.

(f)

Iteraciones del primer *for*:

$$i = 0.$$

$$i = 2.$$

$$i = 4.$$

$$\dots$$

$$i = 2(k - 1).$$

$$2(k - 1) = n^2$$

$$k - 1 = \frac{n^2}{2}$$

$$k = \frac{n^2}{2} + 1.$$

Iteraciones del segundo *for*:

$$j = n.$$

$$j = \frac{3}{4}n.$$

$$j = \frac{1}{2}n.$$

$$j = \frac{1}{4}n.$$

Entonces:

$$T(n) = cte_1 + \sum_{i=1}^{\frac{n^2}{2}+1} \sum_{j=1}^4 cte_2$$

$$T(n) = cte_1 + \sum_{i=1}^{\frac{n^2}{2}+1} 4cte_2$$

$$T(n) = cte_1 + (\frac{n^2}{2} + 1) * 4cte_2$$

$$T(n) = cte_1 + (2n^2 + 4) cte_2.$$

Por lo tanto, para este fragmento de código, el orden de tiempo de ejecución es $O(n^2)$.

Ejercicio 8.

Para cada uno de los algoritmos presentados, calcular el $T(n)$.

(a) Expresar, en función de n , el tiempo de ejecución.

(b) Establecer el orden de dicha función usando notación Big-Oh.

```

1. int c = 1;
   while ( c < n ) {
       algo_de_O(1);
       c = 2 * c;
   }

2. int c = n;
   while ( c > 1 ) {
       algo_de_O(1);
       c = c / 2;
   }

3. public static void calcular(int n) {
    int i, j, r = 0;
    for ( i = 1; i < n; i = i+2 )
        for ( j = 1; j <= i; j++ )
            r = r + 1;
    return r;
}

```

(1)

Iteraciones del *while*:

$c = 1$.
 $c = 2$.
 $c = 4$.
 $\dots$
 $c = 2^{k-1}$.

$2^{k-1} = n - 1$
 $\log_2 2^{k-1} = \log_2 (n - 1)$
 $(k - 1) \log_2 2 = \log_2 (n - 1)$
 $(k - 1) * 1 = \log_2 (n - 1)$
 $k - 1 = \log_2 (n - 1)$
 $k = \log_2 (n - 1) + 1$.

Entonces:

$T(n) = cte_1 + \sum_{c=1}^{\log_2(n-1)+1} cte_2$
 $T(n) = cte_1 + [\log_2 (n - 1) + 1] cte_2 \leq O(\log_2 n)$.

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

$$cte_1 + [\log_2 (n - 1) + 1] cte_2 \leq c \log_2 n.$$

Primer término:

$$cte_1 \leq c \log_2 n$$

$$cte_1 \leq cte_1 \log_2 n.$$

$$c_1 = cte_1; n_0 = 2.$$

Segundo término:

$$cte_2 [\log_2 (n - 1) + 1] \leq c \log_2 n$$

$$cte_2 [\log_2 (n - 1) + 1] \leq cte_2 \log_2 n.$$

$$c_2 = cte_2; n_0 = 2.$$

Entonces:

$$cte_1 + [\log_2 (n - 1) + 1] cte_2 \leq c_1 \log_2 n + c_2 \log_2 n$$

$$cte_1 + [\log_2 (n - 1) + 1] cte_2 \leq (c_1 + c_2) \log_2 n$$

$$cte_1 + [\log_2 (n - 1) + 1] cte_2 \leq (cte_1 + cte_2) \log_2 n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 2.$$

Por lo tanto, $T(n)$ es $O(\log_2 n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 2$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

(2)

Iteraciones del while:

$$c = n.$$

$$c = \frac{n}{2}.$$

$$c = \frac{n}{4}.$$

...

$$c = \frac{n}{2^{k-1}}.$$

$$\frac{n}{2^{k-1}} = 1 + 1$$

$$\frac{n}{2^{k-1}} = 2$$

$$2 * 2^{k-1} = n$$

$$2^k = n$$

$$\log_2 2^k = \log_2 n$$

$$k \log_2 2 = \log_2 n$$

$$k * 1 = \log_2 n$$

$$k = \log_2 n.$$

Entonces:

$$T(n) = cte_1 + \sum_{c=1}^{\log_2 n} cte_2$$

$$T(n) = cte_1 + \log_2 n * cte_2 \leq O(\log_2 n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 \leq c \log_2 n$$

$$cte_1 \leq cte_1 \log_2 n.$$

$$c_1 = cte_1; n_0 = 2.$$

Segundo término:

$$cte_2 \log_2 n \leq c \log_2 n$$

$$cte_2 \log_2 n \leq cte_2 \log_2 n.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$cte_1 + \log_2 n * cte_2 \leq c_1 \log_2 n + c_2 \log_2 n$$

$$cte_1 + \log_2 n * cte_2 \leq (c_1 + c_2) \log_2 n$$

$$cte_1 + \log_2 n * cte_2 \leq (cte_1 + cte_2) \log_2 n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 2.$$

Por lo tanto, $T(n)$ es $O(\log_2 n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 2$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

(3)

Iteraciones del primer for:

$$i = 1.$$

$$i = 3.$$

$$i = 5.$$

$$\dots$$

$$i = 2k - 1.$$

$$2k - 1 = n - 1$$

$$2k = n - 1 + 1$$

$$2k = n$$

$$k = \frac{n}{2}.$$

Iteraciones del segundo for:

j= 1.

j= 2.

j= 3.

...

j= k.

k= i.

Entonces:

$$T(n) = cte_1 + \sum_{i=1}^{\frac{n}{2}} \sum_{j=1}^i cte_2$$

$$T(n) = cte_1 + \sum_{i=1}^{\frac{n}{2}} i * cte_2$$

$$T(n) = cte_1 + cte_2 \sum_{i=1}^{\frac{n}{2}} i$$

$$T(n) = cte_1 + cte_2 \frac{\frac{n}{2}(\frac{n}{2}+1)}{2}$$

$$T(n) = cte_1 + \frac{\frac{n^2}{4} + \frac{n}{2}}{2} cte_2$$

$$T(n) = cte_1 + (\frac{1}{8} n^2 + \frac{1}{4} n) cte_2 \leq O(n^2).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 \leq cn^2$$

$$cte_1 \leq cte_1 n^2.$$

$$c_1 = cte_1; n_0 = 1.$$

Segundo término:

$$cte_2 (\frac{1}{8} n^2 + \frac{1}{4} n) \leq cn^2$$

$$cte_2 (\frac{1}{8} n^2 + \frac{1}{4} n) \leq cte_2 n^2.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$cte_1 + (\frac{1}{8} n^2 + \frac{1}{4} n) cte_2 \leq c_1 n^2 + c_2 n^2$$

$$cte_1 + (\frac{1}{8} n^2 + \frac{1}{4} n) cte_2 \leq (c_1 + c_2) n^2$$

$$cte_1 + (\frac{1}{8} n^2 + \frac{1}{4} n) cte_2 \leq (cte_1 + cte_2) n^2, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n^2)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Ejercicio 9.

(a) *Expresar la función del tiempo de ejecución de cada uno de los siguientes algoritmos, resolverla y calcular el orden.*

```
static public int rec2(int n){
    if (n <= 1)
        return 1;
    else
        return (2 * rec2(n-1));
}

static public int rec1(int n){
    if (n <= 1)
        return 1;
    else
        return (rec1(n-1) + rec1(n-1));
}

static public int rec3(int n){
    if ( n == 0 )
        return 0;
    else {
        if ( n == 1 )
            return 1;
        else
            return (rec3(n-2) * rec3(n-2));
    }
}

static public int potencia_iter(int x, int n){
    int potencia;
    if (n == 0)
        potencia = 1;
    else {
        if (n == 1)
            potencia = x;
        else{
            potencia = x;
            for (int i = 2 ; i <= n ; i++) {
                potencia *= x ;
            }
        }
    }
    return potencia;
}
```

```

static public int potencia_rec( int x, int n){
    if( n == 0 )
        return 1;
    else{
        if( n == 1)
            return x;
        else{
            if ( (n % 2) == 0)
                return potencia_rec (x * x, n / 2 );
            else
                return potencia_rec (x * x, n / 2) * x;
        }
    }
}

```

(1) Algoritmo rec2

$$T(n) = \begin{cases} cte_1, n \leq 1 \\ T(n-1) + cte_2, n > 1 \end{cases}$$

Paso 1:

$$T(n) = T(n-1) + cte_2, \text{ si } n > 1.$$

Paso 2:

$$T(n) = T(n-1-1) + cte_2 + cte_2$$

$$T(n) = T(n-2) + 2cte_2, \text{ si } n > 2.$$

Paso 3:

$$T(n) = T(n-2-1) + cte_2 + 2cte_2$$

$$T(n) = T(n-3) + 3cte_2, \text{ si } n > 3.$$

Paso i (Paso general):

$$T(n) = T(n-i) + icte_2, \text{ si } n > i.$$

$$n - i = 1$$

$$i = n - 1.$$

Entonces:

$$T(n) = T(n - (n-1)) + (n-1) cte_2$$

$$T(n) = T(n - n + 1) + (n-1) cte_2$$

$$T(n) = T(1) + (n-1) cte_2$$

$$T(n) = cte_1 + (n-1) cte_2 \leq O(n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 \leq cn$$

$$cte_1 \leq cte_1 n.$$

$$c_1 = cte_1; n_0 = 1.$$

Segundo término:

$$cte_2 (n - 1) \leq cn$$

$$cte_2 (n - 1) \leq cte_2 n.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$cte_1 + (n - 1) cte_2 \leq c_1 n + c_2 n$$

$$cte_1 + (n - 1) cte_2 \leq (c_1 + c_2) n$$

$$cte_1 + (n - 1) cte_2 \leq (cte_1 + cte_2) n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = cte_1 + (n - 1) cte_2$ y el orden es $O(n)$.

(2) Algoritmo rec1

$$T(n) = \begin{cases} cte_1, & n \leq 1 \\ 2 T(n - 1) + cte_2, & n > 1 \end{cases}$$

Paso 1:

$$T(n) = 2 T(n - 1) + cte_2, \text{ si } n > 1.$$

Paso 2:

$$T(n) = 2 [2 T(n - 1 - 1) + cte_2] + cte_2$$

$$T(n) = 2 [2 T(n - 2) + cte_2] + cte_2$$

$$T(n) = 4 T(n - 2) + 2cte_2 + cte_2$$

$$T(n) = 4 T(n - 2) + 3cte_2, \text{ si } n > 2.$$

Paso 3:

$$T(n) = 4 [2 T(n - 2 - 1) + cte_2] + 3cte_2$$

$$T(n) = 4 [2 T(n - 3) + cte_2] + 3cte_2$$

$$T(n) = 8 T(n - 3) + 4cte_2 + 3cte_2$$

$$T(n) = 8 T(n - 3) + 7cte_2, \text{ si } n > 3.$$

Paso i (Paso general):

$$T(n) = 2^i T(n - i) + (2^i - 1) cte_2, \text{ si } n > i.$$

$$n - i = 1 \\ i = n - 1.$$

Entonces:

$$\begin{aligned} T(n) &= 2^{n-1} T(n - (n - 1)) + (2^{n-1} - 1) cte_2 \\ T(n) &= 2^{n-1} T(n - n + 1) + (2^{n-1} - 1) cte_2 \\ T(n) &= 2^{n-1} T(1) + (2^{n-1} - 1) cte_2 \\ T(n) &= 2^{n-1} cte_1 + (2^{n-1} - 1) cte_2 \leq O(2^n). \end{aligned}$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$\begin{aligned} cte_1 2^{n-1} &\leq c 2^n \\ cte_1 2^{n-1} &\leq cte_1 2^n. \end{aligned}$$

$$c_1 = cte_1; n_0 = 1.$$

Segundo término:

$$\begin{aligned} cte_2 (2^{n-1} - 1) &\leq c 2^n \\ cte_2 (2^{n-1} - 1) &\leq cte_2 2^n. \end{aligned}$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$\begin{aligned} 2^{n-1} cte_1 + (2^{n-1} - 1) cte_2 &\leq c_1 2^n + c_2 2^n \\ 2^{n-1} cte_1 + (2^{n-1} - 1) cte_2 &\leq (c_1 + c_2) 2^n \\ 2^{n-1} cte_1 + (2^{n-1} - 1) cte_2 &\leq (cte_1 + cte_2) 2^n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } \\ n_0 &= 1. \end{aligned}$$

Por lo tanto, $T(n)$ es $O(2^n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 2^{n-1} cte_1 + (2^{n-1} - 1) cte_2$ y el orden es $O(2^n)$.

(3) Algoritmo rec3

$$T(n) = \begin{cases} cte_1, n \leq 1 \\ 2 T(n - 2) + cte_2, n > 1 \end{cases}$$

Paso 1:

$$T(n) = 2 T(n - 2) + cte_2, \text{ si } n > 1.$$

Paso 2:

$$T(n) = 2 [2 T(n - 2 - 2) + cte_2] + cte_2$$

$$T(n) = 2 [2 T(n - 4) + cte_2] + cte_2$$

$$T(n) = 4 T(n - 4) + 2cte_2 + cte_2$$

$$T(n) = 4 T(n - 4) + 3cte_2, \text{ si } n > 2.$$

Paso 3:

$$T(n) = 4 [2 T(n - 4 - 2) + cte_2] + 3cte_2$$

$$T(n) = 4 [2 T(n - 6) + cte_2] + 3cte_2$$

$$T(n) = 8 T(n - 6) + 4cte_2 + 3cte_2$$

$$T(n) = 8 T(n - 6) + 7cte_2, \text{ si } n > 3.$$

Paso i (Paso general):

$$T(n) = 2^i T(n - 2i) + (2^i - 1) cte_2, \text{ si } n > i.$$

$$n - 2i = 1$$

$$2i = n - 1$$

$$i = \frac{n-1}{2}.$$

Entonces:

$$T(n) = 2^{\frac{n-1}{2}} T(n - 2 \frac{n-1}{2}) + (2^{\frac{n-1}{2}} - 1) cte_2$$

$$T(n) = 2^{\frac{n-1}{2}} T(n - (n - 1)) + (2^{\frac{n-1}{2}} - 1) cte_2$$

$$T(n) = 2^{\frac{n-1}{2}} T(n - n + 1) + (2^{\frac{n-1}{2}} - 1) cte_2$$

$$T(n) = 2^{\frac{n-1}{2}} T(1) + (2^{\frac{n-1}{2}} - 1) cte_2$$

$$T(n) = 2^{\frac{n-1}{2}} cte_1 + (2^{\frac{n-1}{2}} - 1) cte_2 \leq O(\sqrt{2^n}).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 2^{\frac{n-1}{2}} \leq c \sqrt{2^n}$$

$$cte_1 2^{\frac{n-1}{2}} \leq cte_1 \sqrt{2^n}.$$

$$c_1 = cte_1; n_0 = 1.$$

Segundo término:

$$cte_2 (2^{\frac{n-1}{2}} - 1) \leq c \sqrt{2^n}$$

$$cte_2 (2^{\frac{n-1}{2}} - 1) \leq cte_2 \sqrt{2^n}.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$2^{\frac{n-1}{2}} cte_1 + (2^{\frac{n-1}{2}} - 1) cte_2 \leq c_1 2^{\frac{n-1}{2}} + c_2 2^{\frac{n-1}{2}}$$

$$2^{\frac{n-1}{2}} cte_1 + (2^{\frac{n-1}{2}} - 1) cte_2 \leq (c_1 + c_2) 2^{\frac{n-1}{2}}$$

$$2^{\frac{n-1}{2}} cte_1 + (2^{\frac{n-1}{2}} - 1) cte_2 \leq (cte_1 + cte_2) 2^{\frac{n-1}{2}}, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \\ \text{con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(\sqrt{2^n})$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 2^{\frac{n-1}{2}} cte_1 + (2^{\frac{n-1}{2}} - 1) cte_2$ y el orden es $O(\sqrt{2^n})$.

(4) Algoritmo potencia iter

$$T(n) = cte_1 + \sum_{i=2}^n cte_2$$

$$T(n) = cte_1 + (n - 1) cte_2 \leq O(n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 \leq cn$$

$$cte_1 \leq cte_1 n.$$

$$c_1 = cte_1; n_0 = 1.$$

Segundo término:

$$cte_2 (n - 1) \leq cn$$

$$cte_2 (n - 1) \leq cte_2 n.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$cte_1 + (n - 1) cte_2 \leq c_1 n + c_2 n$$

$$cte_1 + (n - 1) cte_2 \leq (c_1 + c_2) n$$

$$cte_1 + (n - 1) cte_2 \leq (cte_1 + cte_2) n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = cte_1 + (n - 1) cte_2$ y el orden es $O(n)$.

(5) Algoritmo potencia rec

$$T(n) = \begin{cases} cte_1, n \leq 1 \\ T(\frac{n}{2}) + cte_2, n > 1 \end{cases}$$

Paso 1:

$$T(n) = T(\frac{n}{2}) + cte_2, \text{ si } n > 1.$$

Paso 2:

$$T(n) = T(\frac{n}{2}) + cte_2 + cte_2$$

$$T(n) = T(\frac{n}{4}) + 2cte_2, \text{ si } n > 2.$$

Paso 3:

$$T(n) = T(\frac{n}{4}) + cte_2 + 2cte_2$$

$$T(n) = T(\frac{n}{8}) + 3cte_2, \text{ si } n > 3.$$

Paso i (Paso general):

$$T(n) = T(\frac{n}{2^i}) + i cte_2, \text{ si } n > i.$$

$$\frac{n}{2^i} = 1$$

$$1 * 2^i = n$$

$$2^i = n$$

$$i \log_2 2 = \log_2 n$$

$$i * 1 = \log_2 n$$

$$i = \log_2 n.$$

Entonces:

$$T(n) = T(\frac{n}{2^i}) + i cte_2$$

$$T(n) = T(\frac{n}{2^{\log_2 n}}) + \log_2 n * cte_2$$

$$T(n) = T(\frac{n}{n}) + \log_2 n * cte_2$$

$$T(n) = T(1) + \log_2 n * cte_2$$

$$T(n) = cte_1 + \log_2 n * cte_2 \leq O(\log_2 n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$cte_1 \leq c \log_2 n$$

$$cte_1 \leq cte_1 \log_2 n.$$

$$c_1 = cte_1; n_0 = 2.$$

Segundo término:

$$cte_2 \log_2 n \leq c \log_2 n$$

$$cte_2 \log_2 n \leq cte_2 \log_2 n.$$

$$c_2 = cte_2; n_0 = 1.$$

Entonces:

$$cte_1 + \log_2 n * cte_2 \leq c_1 \log_2 n + c_2 \log_2 n$$

$$cte_1 + \log_2 n * cte_2 \leq (c_1 + c_2) \log_2 n$$

$$cte_1 + \log_2 n * cte_2 \leq (cte_1 + cte_2) \log_2 n, \text{ con } c = cte_1 + cte_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(\log_2 n)$, ya que existen constantes $c = cte_1 + cte_2$ y $n_0 = 2$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = cte_1 + \log_2 n * cte_2$ y el orden es $O(\log_2 n)$.

(b) Comparar el tiempo de ejecución del método “rec2” con el del método “rec1”.

El tiempo de ejecución del método “rec2” es menor al del método “rec1”.

(c) Implementar un algoritmo más eficiente que el del método “rec3” (es decir, que el $T(n)$ sea menor).

```
static public int rec3Mejorado(int n) {
    return n%2
}
```

Este algoritmo es más eficiente que el del método “rec3”, ya que el orden de tiempo de ejecución es $O(1)$.

Ejercicio 10.

(a) Resolver las siguientes recurrencias.

(b) Calcular el $O(n)$. Justificar usando la definición de Big-Oh.

1.

$$T(n) = \begin{cases} 2, & n = 1 \\ T(n-1) + n, & n \geq 2 \end{cases}$$

2.

$$T(n) = \begin{cases} 1, & n = 1 \\ 2T\left(\frac{n}{4}\right) + \sqrt{n}, & n \geq 2 \end{cases}$$

2.

$$T(n) = \begin{cases} 2, & n = 1 \\ T(n-1) + \frac{n}{2}, & n \geq 2 \end{cases}$$

3.

$$T(n) = \begin{cases} 1, & n = 1 \\ 4T\left(\frac{n}{2}\right) + n^2, & n \geq 2 \end{cases}$$

$$(1) T(n) = \begin{cases} 2, & n = 1 \\ T(n-1) + n, & n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = T(n-1) + n, \text{ si } n \geq 2.$$

Paso 2:

$$\begin{aligned} T(n) &= T(n-1-1) + (n-1) + n \\ T(n) &= T(n-2) + (n-1) + n, \text{ si } n \geq 3. \end{aligned}$$

Paso 3:

$$\begin{aligned} T(n) &= T(n-2-1) + (n-2) + (n-1) + n \\ T(n) &= T(n-3) + (n-2) + (n-1) + n, \text{ si } n \geq 4. \end{aligned}$$

Paso i (Paso general):

$$\begin{aligned} T(n) &= T(n-i) + (n-i+1) + (n-i+2) + \dots + n \\ T(n) &= T(n-i) + \sum_{k=n-i+1}^n k, \text{ si } n \geq i+1. \end{aligned}$$

$$\begin{aligned} n-i &= 1 \\ i &= n-1. \end{aligned}$$

Entonces:

$$\begin{aligned} T(n) &= T(n-(n-1)) + \sum_{k=n-(n-1)+1}^n k \\ T(n) &= T(n-n+1) + \sum_{k=n-n+1+1}^n k \\ T(n) &= T(1) + \sum_{k=2}^n k \\ T(n) &= 2 + \sum_{k=1}^n k - 1 \end{aligned}$$

$$T(n) = \frac{n(n+1)}{2} + 1$$

$$T(n) = \frac{n^2 + n}{2} + 1$$

$$T(n) = \frac{1}{2} n^2 + \frac{1}{2} n + 1 \leq O(n^2).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$\frac{1}{2} n^2 \leq c n^2$$

$$\frac{1}{2} n^2 \leq \frac{1}{2} n^2.$$

$$c_1 = \frac{1}{2}; n_0 = 1.$$

Segundo término:

$$\frac{1}{2} n \leq c n^2$$

$$\frac{1}{2} n \leq \frac{1}{2} n^2.$$

$$c_2 = \frac{1}{2}; n_0 = 1.$$

Tercer término:

$$1 \leq c n^2$$

$$1 \leq 1 n^2$$

$$1 \leq n^2.$$

$$c_3 = 1; n_0 = 1.$$

Entonces:

$$\frac{1}{2} n^2 + \frac{1}{2} n + 1 \leq c_1 n^2 + c_2 n^2 + c_3 n^2$$

$$\frac{1}{2} n^2 + \frac{1}{2} n + 1 \leq (c_1 + c_2 + c_3) n^2$$

$$\frac{1}{2} n^2 + \frac{1}{2} n + 1 \leq \left(\frac{1}{2} + \frac{1}{2} + 1\right) n^2$$

$$\frac{1}{2} n^2 + \frac{1}{2} n + 1 \leq 2 n^2, \text{ con } c = 2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n^2)$, ya que existen constantes $c = 2$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

$$(2) T(n) = \begin{cases} 2, & n = 1 \\ T(n-1) + \frac{n}{2}, & n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = T(n - 1) + \frac{n}{2}, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = T(n - 1 - 1) + \frac{n-1}{2} + \frac{n}{2}$$

$$T(n) = T(n - 2) + \frac{n-1}{2} + \frac{n}{2}, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = T(n - 2 - 1) + \frac{n-2}{2} + \frac{n-1}{2} + \frac{n}{2}$$

$$T(n) = T(n - 3) + \frac{n-2}{2} + \frac{n-1}{2} + \frac{n}{2}, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = T(n - i) + \frac{n-i+1}{2} + \frac{n-i+2}{2} + \dots + \frac{n}{2}$$

$$T(n) = T(n - i) + \frac{1}{2} [(n - i + 1) + (n - i + 2) + \dots + n]$$

$$T(n) = T(n - i) + \frac{1}{2} \sum_{k=n-i+1}^n k, \text{ si } n \geq i + 1.$$

$$n - i = 1$$

$$i = n - 1.$$

Entonces:

$$T(n) = T(n - (n - 1)) + \frac{1}{2} \sum_{k=n-(n-1)+1}^n k$$

$$T(n) = T(n - n + 1) + \frac{1}{2} \sum_{k=n-n+1+1}^n k$$

$$T(n) = T(1) + \frac{1}{2} \sum_{k=2}^n k$$

$$T(n) = 2 + \frac{1}{2} (\sum_{k=1}^n k - 1)$$

$$T(n) = 2 + \frac{1}{2} \left[\frac{n(n+1)}{2} - 1 \right]$$

$$T(n) = 2 + \frac{1}{2} \left(\frac{n^2+n}{2} - 1 \right)$$

$$T(n) = 2 + \frac{1}{2} \left(\frac{1}{2} n^2 + \frac{1}{2} n - 1 \right)$$

$$T(n) = 2 + \frac{1}{4} n^2 + \frac{1}{4} n - \frac{1}{2}$$

$$T(n) = \frac{1}{4} n^2 + \frac{1}{4} n + \frac{3}{4} \leq O(n^2).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$\frac{1}{4} n^2 \leq c n^2$$

$$\frac{1}{4} n^2 \leq \frac{1}{4} n^2.$$

$$c_1 = \frac{1}{4}; n_0 = 1.$$

Segundo término:

$$\frac{1}{4}n \leq cn^2$$

$$\frac{1}{4}n \leq \frac{1}{4}n^2.$$

$$c_2 = \frac{1}{4}; n_0 = 1.$$

Tercer término:

$$\frac{3}{4} \leq cn^2$$

$$\frac{3}{4} \leq \frac{3}{4}n^2.$$

$$c_3 = \frac{3}{4}; n_0 = 1.$$

Entonces:

$$\frac{1}{4}n^2 + \frac{1}{4}n + \frac{3}{4} \leq c_1n^2 + c_2n^2 + c_3n^2$$

$$\frac{1}{4}n^2 + \frac{1}{4}n + \frac{3}{4} \leq (c_1 + c_2 + c_3)n^2$$

$$\frac{1}{4}n^2 + \frac{1}{4}n + \frac{3}{4} \leq \left(\frac{1}{4} + \frac{1}{4} + \frac{3}{4}\right)n^2$$

$$\frac{1}{4}n^2 + \frac{1}{4}n + \frac{3}{4} \leq \frac{5}{4}n^2, \text{ con } c = \frac{5}{4} \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n^2)$, ya que existen constantes $c = \frac{5}{4}$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

$$(3) T(n) = \begin{cases} 1, & n = 1 \\ 2 T(\frac{n}{4}) + \sqrt{n}, & n \geq 2. \end{cases}$$

Paso 1:

$$T(n) = 2 T(\frac{n}{4}) + \sqrt{n}, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = 2 [2 T(\frac{n}{4}) + \sqrt{\frac{n}{4}}] + \sqrt{n}$$

$$T(n) = 2 [2 T(\frac{n}{16}) + \frac{\sqrt{n}}{2}] + \sqrt{n}$$

$$T(n) = 4 T(\frac{n}{16}) + \sqrt{n} + \sqrt{n}$$

$$T(n) = 4 T(\frac{n}{16}) + 2 \sqrt{n}, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = 4 \left[2 T\left(\frac{n}{4}\right) + \sqrt{\frac{n}{16}} \right] + 2 \sqrt{n}$$

$$T(n) = 4 \left[2 T\left(\frac{n}{64}\right) + \frac{\sqrt{n}}{4} \right] + 2 \sqrt{n}$$

$$T(n) = 8 T\left(\frac{n}{64}\right) + \sqrt{n} + 2 \sqrt{n}$$

$$T(n) = 8 T\left(\frac{n}{64}\right) + 3 \sqrt{n}, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = 2^i T\left(\frac{n}{4^i}\right) + i \sqrt{n}, \text{ si } n \geq i + 1.$$

$$\frac{n}{4^i} = 1$$

$$1 * 4^i = n$$

$$4^i = n$$

$$\log_4 4^i = \log_4 n$$

$$i \log_4 4 = \log_4 n$$

$$i * 1 = \log_4 n$$

$$i = \log_4 n.$$

$$\frac{n}{2^{2i}} = 1$$

$$1 * 2^{2i} = n$$

$$2^{2i} = n$$

$$\log_2 2^{2i} = \log_2 n$$

$$2i \log_2 2 = \log_2 n$$

$$2i * 1 = \log_2 n$$

$$2i = \log_2 n$$

$$i = \frac{\log_2 n}{2}.$$

Entonces:

$$T(n) = 2^{\log_4 n} T\left(\frac{n}{4^{\log_4 n}}\right) + \log_4 n * \sqrt{n}$$

$$T(n) = 2^{\frac{\log_2 n}{2}} T\left(\frac{n}{n}\right) + \log_4 n * \sqrt{n}$$

$$T(n) = \sqrt{2^{\log_2 n}} T(1) + \log_4 n * \sqrt{n}$$

$$T(n) = \sqrt{n} * 1 + \log_4 n * \sqrt{n}$$

$$T(n) = \sqrt{n} + \log_4 n * \sqrt{n} \leq O(\log_4 n * \sqrt{n}).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$\sqrt{n} \leq c \log_4 n * \sqrt{n}$$

$$\sqrt{n} \leq 1 \log_4 n * \sqrt{n}$$

$$\sqrt{n} \leq \log_4 n * \sqrt{n}.$$

$$c_1 = 1; n_0 = 4.$$

Segundo término:

$$\log_4 n * \sqrt{n} \leq c \log_4 n * \sqrt{n}$$

$$\log_4 n * \sqrt{n} \leq 1 \log_4 n * \sqrt{n}$$

$$\log_4 n * \sqrt{n} \leq \log_4 n * \sqrt{n}.$$

$$c_2 = 1; n_0 = 1.$$

Entonces:

$$\sqrt{n} + \log_4 n * \sqrt{n} \leq c_1 \log_4 n * \sqrt{n} + c_2 \log_4 n * \sqrt{n}$$

$$\sqrt{n} + \log_4 n * \sqrt{n} \leq (c_1 + c_2) \log_4 n * \sqrt{n}$$

$$\sqrt{n} + \log_4 n * \sqrt{n} \leq (1 + 1) \log_4 n * \sqrt{n}$$

$$\sqrt{n} + \log_4 n * \sqrt{n} \leq 2 \log_4 n * \sqrt{n}$$

Por lo tanto, $T(n)$ es $O(\log_4 n * \sqrt{n})$, ya que existen constantes $c = 2$ y $n_0 = 4$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

$$(4) T(n) = \begin{cases} 1, & n = 1 \\ 4 T(\frac{n}{2}) + n^2, & n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = 4 T(\frac{n}{2}) + n^2, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = 4 [4 T(\frac{n}{2}) + (\frac{n}{2})^2] + n^2$$

$$T(n) = 4 [4 T(\frac{n}{4}) + \frac{n^2}{4}] + n^2$$

$$T(n) = 16 T(\frac{n}{4}) + n^2 + n^2$$

$$T(n) = 16 T(\frac{n}{4}) + 2n^2, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = 16 [4 T(\frac{n}{4}) + (\frac{n}{4})^2] + 2n^2$$

$$T(n) = 16 [4 T(\frac{n}{8}) + \frac{n^2}{16}] + 2n^2$$

$$T(n) = 64 T(\frac{n}{8}) + n^2 + 2n^2$$

$$T(n) = 64 T(\frac{n}{8}) + 3n^2, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = 4^i T\left(\frac{n}{2^i}\right) + in^2, \text{ si } n \geq i + 1.$$

$$\frac{n}{2^i} = 1$$

$$1 * 2^i = n$$

$$2^i = n$$

$$\log_2 2^i = \log_2 n$$

$$i \log_2 2 = \log_2 n$$

$$i * 1 = \log_2 n$$

$$i = \log_2 n.$$

Entonces:

$$T(n) = 4^{\log_2 n} T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n * n^2$$

$$T(n) = (2 * 2)^{\log_2 n} T\left(\frac{n}{n}\right) + \log_2 n * n^2$$

$$T(n) = 2^{\log_2 n} 2^{\log_2 n} T(1) + \log_2 n * n^2$$

$$T(n) = nn * 1 + \log_2 n * n^2$$

$$T(n) = n^2 + \log_2 n * n^2 \leq O(\log_2 n * n^2).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$n^2 \leq c \log_2 n * n^2$$

$$n^2 \leq 1 \log_2 n * n^2$$

$$n^2 \leq \log_2 n * n^2.$$

$$c_1 = 1; n_0 = 2.$$

Segundo término:

$$\log_2 n * \sqrt{n} \leq c \log_2 n * \sqrt{n}$$

$$\log_2 n * \sqrt{n} \leq 1 \log_2 n * \sqrt{n}$$

$$\log_2 n * \sqrt{n} \leq \log_2 n * \sqrt{n}.$$

$$c_2 = 1; n_0 = 1.$$

Entonces:

$$n^2 + \log_2 n * n^2 \leq c_1 \log_2 n * n^2 + c_2 \log_2 n * n^2$$

$$n^2 + \log_2 n * n^2 \leq (c_1 + c_2) \log_2 n * n^2$$

$$n^2 + \log_2 n * n^2 \leq (1 + 1) \log_2 n * n^2$$

$$n^2 + \log_2 n * n^2 \leq 2 \log_2 n * n^2$$

Por lo tanto, $T(n)$ es $O(\log_2 n * n^2)$, ya que existen constantes $c = 2$ y $n_0 = 2$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Ejercicio 11.

Calcular el tiempo de ejecución de los métodos *buscarLineal* y *buscarDicotomica* de la clase *BuscadorEnArrayOrdenado*. Comparar el tiempo con los valores obtenidos, empíricamente, en el Ejercicio 2.

(1) buscarLineal

$$T(n) = cte_1 + \sum_{i=1}^n cte_2$$

$$T(n) = cte_1 + n cte_2 \leq O(n).$$

(2) buscarDicotomica

$$T(n) = \begin{cases} cte_1, n \leq 1 \\ T(\frac{n}{2}) + cte_2, n > 1 \end{cases}$$

Paso 1:

$$T(n) = T(\frac{n}{2}) + cte_2, \text{ si } n > 1.$$

Paso 2:

$$T(n) = T(\frac{n}{2}) + cte_2 + cte_2$$

$$T(n) = T(\frac{n}{4}) + 2cte_2, \text{ si } n > 2.$$

Paso 3:

$$T(n) = T(\frac{n}{2}) + cte_2 + 2cte_2$$

$$T(n) = T(\frac{n}{8}) + 3cte_2, \text{ si } n > 3.$$

Paso i (Paso general):

$$T(n) = T(\frac{n}{2^i}) + i cte_2, \text{ si } n > i.$$

$$\frac{n}{2^i} = 1$$

$$1 * 2^i = n$$

$$2^i = n$$

$$\log_2 2^i = \log_2 n$$

$$i \log_2 2 = \log_2 n$$

$$i * 1 = \log_2 n$$

$$i = \log_2 n.$$

Entonces:

$$T(n) = T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n * cte_2$$

$$T(n) = T\left(\frac{n}{n}\right) + \log_2 n * cte_2$$

$$T(n) = T(1) + \log_2 n * cte_2$$

$$T(n) = cte_1 + \log_2 n * cte_2 \leq O(\log_2 n).$$

Ejercicio 12.

Calcular el tiempo de ejecución de *procesarMovimientos* y *procesarMovimientosOptimizado* del Ejercicio 1. Comparar el tiempo con los valores obtenidos empíricamente.

(1) *procesarMovimientos*

$$T(n) = cte_1 + \sum_{i=1}^n (cte_2 + \sum_{i=1}^{n+1} cte_3)$$

$$T(n) = cte_1 + \sum_{i=1}^n [cte_2 + (n+1) cte_3]$$

$$T(n) = cte_1 + n [cte_2 + (n+1) cte_3]$$

$$T(n) = cte_1 + n cte_2 + n^2 cte_3 + n cte_3$$

$$T(n) = cte_1 + n (cte_2 + cte_3) + n^2 cte_3 \leq O(n^2).$$

(2) *procesarMovimientosOptimizado*

$$T(n) = cte_1 + \sum_{i=1}^n cte_2 + \sum_{i=1}^n cte_3$$

$$T(n) = cte_1 + n cte_2 + n cte_3$$

$$T(n) = cte_1 + n (cte_2 + cte_3) \leq O(n).$$

Ejercicio 13.

Resolver las recurrencias y calcular el orden. Para cada recurrencia, se muestra, a modo de ejemplo, el código correspondiente.

<pre>int recursivo(int n){ if (n <= 1) return 1; else return (recursivo (n-1)); }</pre> $T(n) = \begin{cases} 1, n \leq 1 \\ T(n-1) + c, n \geq 2 \end{cases}$	<pre>int recursivo(int n){ if (n == 1) return 1; else return (recursivo (n/2)); }</pre> $T(n) = \begin{cases} 1, n = 1 \\ T(n/2) + c, n \geq 2 \end{cases}$
<pre>int recur (int n){ if (n == 1) return 1; else return (recur(n/2)+recur(n/2)); }</pre> $T(n) = \begin{cases} 1, n = 1 \\ 2T(n/2) + c, n \geq 2 \end{cases}$	<pre>int recursivo(int n){ if (n <= 5) return 1; else return (recursivo (n-5)); }</pre> $T(n) = \begin{cases} 1, n \leq 5 \\ T(n-5) + c, n \geq 6 \end{cases}$
<pre>int recur (int n){ if (n == 1) return 1; else return (recur(n-1)+recur(n-1)); }</pre> $T(n) = \begin{cases} 1, n = 1 \\ 2T(n-1) + c, n \geq 2 \end{cases}$	<pre>int recursivo(int n){ if (n <= 7) return 1; else return (recursivo (n/8)); }</pre> $T(n) = \begin{cases} 1, n \leq 7 \\ T(n/8) + c, n \geq 8 \end{cases}$

$$(a) T(n) = \begin{cases} 1, n \leq 1 \\ T(n-1) + cte, n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = T(n-1) + cte, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = T(n-1-1) + cte + cte$$

$$T(n) = T(n-2) + 2cte, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = T(n-2-1) + cte + 2cte$$

$$T(n) = T(n-3) + 3cte, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = T(n-i) + i cte, \text{ si } n \geq i+1.$$

$$n-i = 1$$

$$i = n-1.$$

Entonces:

$$T(n) = T(n - (n - 1)) + (n - 1) \text{ cte}$$

$$T(n) = T(n - n + 1) + (n - 1) \text{ cte}$$

$$T(n) = T(1) + (n - 1) \text{ cte}$$

$$T(n) = 1 + (n - 1) \text{ cte} \leq O(n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$1 \leq cn$$

$$1 \leq 1n$$

$$1 \leq n.$$

$$c_1 = 1; n_0 = 1.$$

Segundo término:

$$\text{cte}(n - 1) \leq cn$$

$$\text{cte}(n - 1) \leq \text{cte} n.$$

$$c_2 = \text{cte}; n_0 = 1.$$

Entonces:

$$1 + (n - 1) \text{ cte} \leq c_1 n + c_2 n$$

$$1 + (n - 1) \text{ cte} \leq (c_1 + c_2) n$$

$$1 + (n - 1) \text{ cte} \leq (1 + \text{cte}) n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 1 + (n - 1) \text{ cte}$ y el orden es $O(n)$.

$$(b) T(n) = \begin{cases} 1, n = 1 \\ T(\frac{n}{2}) + \text{cte}, n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = T(\frac{n}{2}) + \text{cte}, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = T(\frac{n}{2}) + \text{cte} + \text{cte}$$

$$T(n) = T\left(\frac{n}{4}\right) + 2\text{cte}, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = T\left(\frac{n}{4}\right) + \text{cte} + 2\text{cte}$$

$$T(n) = T\left(\frac{n}{8}\right) + 3\text{cte}, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = T\left(\frac{n}{2^i}\right) + i \text{ cte}, \text{ si } n \geq i + 1.$$

$$\frac{n}{2^i} = 1$$

$$1 * 2^i = n$$

$$2^i = n$$

$$i \log_2 2 = \log_2 n$$

$$i * 1 = \log_2 n$$

$$i = \log_2 n.$$

Entonces:

$$T(n) = T\left(\frac{n}{2^{\log_2 n}}\right) + \log_2 n * \text{cte}$$

$$T(n) = T\left(\frac{n}{n}\right) + \log_2 n * \text{cte}$$

$$T(n) = T(1) + \log_2 n * \text{cte}$$

$$T(n) = 1 + \log_2 n * \text{cte} \leq O(\log_2 n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$1 \leq c \log_2 n$$

$$1 \leq 1 \log_2 n$$

$$1 \leq \log_2 n.$$

$$c_1 = 1; n_0 = 2.$$

Segundo término:

$$\text{cte} \log_2 n \leq c \log_2 n$$

$$\text{cte} \log_2 n \leq \text{cte} \log_2 n.$$

$$c_2 = \text{cte}; n_0 = 1.$$

Entonces:

$$1 + \log_2 n * \text{cte} \leq c_1 \log_2 n + c_2 \log_2 n$$

$$1 + \log_2 n * \text{cte} \leq (c_1 + c_2) \log_2 n$$

$$1 + \log_2 n * \text{cte} \leq (1 + \text{cte}) \log_2 n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 2.$$

Por lo tanto, $T(n)$ es $O(\log_2 n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 2$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 1 + \log_2 n * \text{cte}$ y el orden es $O(\log_2 n)$.

$$(c) T(n) = \begin{cases} 1, n = 1 \\ 2 T(\frac{n}{2}) + \text{cte}, n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = 2 T(\frac{n}{2}) + \text{cte}, \text{ si } n \geq 2.$$

Paso 2:

$$\begin{aligned} T(n) &= 2 [2 T(\frac{n}{2}) + \text{cte}] + \text{cte} \\ T(n) &= 2 [2 T(\frac{n}{4}) + \text{cte}] + \text{cte} \\ T(n) &= 4 T(\frac{n}{4}) + 2\text{cte} + \text{cte} \\ T(n) &= 4 T(\frac{n}{4}) + 3\text{cte}, \text{ si } n \geq 3. \end{aligned}$$

Paso 3:

$$\begin{aligned} T(n) &= 4 [2 T(\frac{n}{4}) + \text{cte}] + 3\text{cte} \\ T(n) &= 4 [2 T(\frac{n}{8}) + \text{cte}] + 3\text{cte} \\ T(n) &= 8 T(\frac{n}{8}) + 4\text{cte} + 3\text{cte} \\ T(n) &= 8 T(\frac{n}{8}) + 7\text{cte}, \text{ si } n \geq 4. \end{aligned}$$

Paso i (Paso general):

$$T(n) = 2^i T(\frac{n}{2^i}) + (2^i - 1) \text{cte}, \text{ si } n \geq i + 1.$$

$$\begin{aligned} \frac{n}{2^i} &= 1 \\ 1 * 2^i &= n \\ 2^i &= n \\ i \log_2 2 &= \log_2 n \\ i * 1 &= \log_2 n \\ i &= \log_2 n. \end{aligned}$$

Entonces:

$$T(n) = 2^{\log_2 n} T(\frac{n}{2^{\log_2 n}}) + (2^{\log_2 n} - 1) \text{cte}$$

$$T(n) = n T\left(\frac{n}{n}\right) + (n - 1) \text{ cte}$$

$$T(n) = n T(1) + (n - 1) \text{ cte}$$

$$T(n) = n * 1 + (n - 1) \text{ cte}$$

$$T(n) = n + (n - 1) \text{ cte} \leq O(n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$n \leq cn$$

$$n \leq 1n$$

$$n \leq n.$$

$$c_1 = 1; n_0 = 1.$$

Segundo término:

$$\text{cte} (n - 1) \leq cn$$

$$\text{cte} (n - 1) \leq \text{cte} n.$$

$$c_2 = \text{cte}; n_0 = 1.$$

Entonces:

$$n + (n - 1) \text{ cte} \leq c_1 n + c_2 n$$

$$n + (n - 1) \text{ cte} \leq (c_1 + c_2) n$$

$$n + (n - 1) \text{ cte} \leq (1 + \text{cte}) n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = n + (n - 1) \text{ cte}$ y el orden es $O(n)$.

$$(d) T(n) = \begin{cases} 1, & n \leq 5 \\ T(n - 5) + \text{cte}, & n \geq 6 \end{cases}$$

Paso 1:

$$T(n) = T(n - 5) + \text{cte}, \text{ si } n \geq 6.$$

Paso 2:

$$T(n) = T(n - 5 - 5) + \text{cte} + \text{cte}$$

$$T(n) = T(n - 10) + 2\text{cte}, \text{ si } n \geq 7.$$

Paso 3:

$$T(n) = T(n - 10 - 5) + \text{cte} + 2\text{cte}$$

$$T(n) = T(n - 15) + 3\text{cte}, \text{ si } n \geq 8.$$

Paso i (Paso general):

$$T(n) = T(n - 5i) + i \text{ cte}, \text{ si } n \geq i + 5.$$

$$n - 5i = 5$$

$$5i = n - 5$$

$$i = \frac{n-5}{5}.$$

Entonces:

$$T(n) = T(n - 5 \frac{n-5}{5}) + \frac{n-1}{5} \text{ cte}$$

$$T(n) = T(n - (n - 5)) + \frac{n-1}{5} \text{ cte}$$

$$T(n) = T(n - n + 5) + \frac{n-1}{5} \text{ cte}$$

$$T(n) = T(5) + \frac{n-1}{5} \text{ cte}$$

$$T(n) = 1 + \frac{n-5}{5} \text{ cte} \leq O(n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$1 \leq cn$$

$$1 \leq 1n$$

$$1 \leq n.$$

$$c_1 = 1; n_0 = 1.$$

Segundo término:

$$\text{cte } \frac{n-5}{5} \leq cn$$

$$\text{cte } \frac{n-5}{5} \leq \text{cte } n.$$

$$c_2 = \text{cte}; n_0 = 1.$$

Entonces:

$$1 + \frac{n-5}{5} \text{ cte} \leq c_1 n + c_2 n$$

$$1 + \frac{n-5}{5} \text{ cte} \leq (c_1 + c_2) n$$

$$1 + \frac{n-5}{5} \text{ cte} \leq (1 + \text{cte}) n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 1 + \frac{n-5}{5}$ cte y el orden es $O(n)$.

$$(e) T(n) = \begin{cases} 1, n \leq 1 \\ 2 T(n-1) + cte, n \geq 2 \end{cases}$$

Paso 1:

$$T(n) = 2 T(n-1) + cte, \text{ si } n \geq 2.$$

Paso 2:

$$T(n) = 2 [2 T(n-1-1) + cte] + cte$$

$$T(n) = 2 [2 T(n-2) + cte] + cte$$

$$T(n) = 4 T(n-2) + 2cte + cte$$

$$T(n) = 4 T(n-2) + 3cte, \text{ si } n \geq 3.$$

Paso 3:

$$T(n) = 4 [2 T(n-2-1) + cte] + 3cte$$

$$T(n) = 4 [2 T(n-3) + cte] + 3cte$$

$$T(n) = 8 T(n-3) + 4cte + 3cte$$

$$T(n) = 8 T(n-3) + 7cte, \text{ si } n \geq 4.$$

Paso i (Paso general):

$$T(n) = 2^i T(n-i) + (2^i - 1) cte, \text{ si } n \geq i+1.$$

$$n - i = 1$$

$$i = n - 1.$$

Entonces:

$$T(n) = 2^{n-1} T(n - (n-1)) + (2^{n-1} - 1) cte$$

$$T(n) = 2^{n-1} T(n - n + 1) + (2^{n-1} - 1) cte$$

$$T(n) = 2^{n-1} T(1) + (2^{n-1} - 1) cte$$

$$T(n) = 2^{n-1} * 1 + (2^{n-1} - 1) cte$$

$$T(n) = 2^{n-1} + (2^{n-1} - 1) cte \leq O(2^n).$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$2^{n-1} \leq c 2^n$$

$$2^{n-1} \leq 1 * 2^n$$

$$2^{n-1} \leq 2^n.$$

$$c_1 = 1; n_0 = 1.$$

Segundo término:

$$\text{cte} (2^{n-1} - 1) \leq c 2^n$$

$$\text{cte} (2^{n-1} - 1) \leq \text{cte} 2^n.$$

$$c_2 = \text{cte}_2; n_0 = 1.$$

Entonces:

$$2^{n-1} + (2^{n-1} - 1) \text{cte} \leq c_1 2^n + c_2 2^n$$

$$2^{n-1} + (2^{n-1} - 1) \text{cte} \leq (c_1 + c_2) 2^n$$

$$2^{n-1} + (2^{n-1} - 1) \text{cte} \leq (1 + \text{cte}) 2^n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

Por lo tanto, $T(n)$ es $O(2^n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 1$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 2^{n-1} + (2^{n-1} - 1) \text{cte}$ y el orden es $O(2^n)$.

$$(f) T(n) = \begin{cases} 1, & n \leq 7 \\ T(\frac{n}{8}) + \text{cte}, & n \geq 8 \end{cases}$$

Paso 1:

$$T(n) = T(\frac{n}{8}) + \text{cte}, \text{ si } n \geq 8.$$

Paso 2:

$$T(n) = T(\frac{\frac{n}{8}}{8}) + \text{cte} + \text{cte}$$

$$T(n) = T(\frac{n}{64}) + 2\text{cte}, \text{ si } n \geq 9.$$

Paso 3:

$$T(n) = T(\frac{\frac{n}{64}}{8}) + \text{cte} + 2\text{cte}$$

$$T(n) = T(\frac{n}{512}) + 3\text{cte}, \text{ si } n \geq 10.$$

Paso i (Paso general):

$$T(n) = T(\frac{n}{8^i}) + i \text{cte}, \text{ si } n \geq i + 7.$$

$$\frac{n}{8^i} = 7$$

$$7 * 8^i = n$$

$$\log_8 (7 * 8^i) = \log_8 n$$

$$\begin{aligned}
\log_8 7 + \log_8 8^i &= \log_8 n \\
\log_8 7 + i \log_8 8 &= \log_8 n \\
\log_8 7 + i * 1 &= \log_8 n \\
\log_8 7 + i &= \log_8 n \\
i &= \log_8 n - \log_8 7 \\
i &= \log_8 \frac{n}{7}.
\end{aligned}$$

Entonces:

$$\begin{aligned}
T(n) &= T\left(\frac{n}{8^{\log_8 7}}\right) + \log_8 \frac{n}{7} * \text{cte} \\
T(n) &= T\left(\frac{n}{8^{\log_8 7}}\right) + \log_8 \frac{n}{7} * \text{cte} \\
T(n) &= T\left(\frac{n}{7}\right) + \log_8 \frac{n}{7} * \text{cte} \\
T(n) &= T(7) + \log_8 \frac{n}{7} * \text{cte} \\
T(n) &= 1 + \log_8 \frac{n}{7} * \text{cte} \leq O(\log_8 n).
\end{aligned}$$

¿Existen constantes $c > 0$ y $n_0 > 0$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$?

Primer término:

$$\begin{aligned}
1 &\leq c \log_8 n \\
1 &\leq 1 \log_8 n \\
1 &\leq \log_8 n.
\end{aligned}$$

$$c_1 = 1; n_0 = 8.$$

Segundo término:

$$\begin{aligned}
\text{cte} \log_8 \frac{n}{7} &\leq c \log_8 n \\
\text{cte} \log_8 \frac{n}{7} &\leq \text{cte} \log_8 n.
\end{aligned}$$

$$c_2 = \text{cte}; n_0 = 1.$$

Entonces:

$$\begin{aligned}
1 + \log_8 \frac{n}{7} * \text{cte} &\leq c_1 \log_8 n + c_2 \log_8 n \\
1 + \log_8 \frac{n}{7} * \text{cte} &\leq (c_1 + c_2) \log_8 n \\
1 + \log_8 \frac{n}{7} * \text{cte} &\leq (1 + \text{cte}) \log_8 n, \text{ con } c = 1 + \text{cte} \text{ para todo } n \geq n_0, \text{ con } n_0 = 8.
\end{aligned}$$

Por lo tanto, $T(n)$ es $O(\log_8 n)$, ya que existen constantes $c = 1 + \text{cte}$ y $n_0 = 8$ tales que $f(n) \leq c g(n)$ para todo $n \geq n_0$.

Por lo tanto, la función de tiempo de ejecución es $T(n) = 1 + \log_8 \frac{n}{7} * \text{cte}$ y el orden es $O(\log_8 n)$.

Ejercicio 14.

Considerar el siguiente fragmento de código:

```
int count = 0; int n = a.length;
    for (int i = 1; i <= n; i = i*2) {
        for (int j = 0; j < n; j += n/2) {
            a[j]++;
        }
    }
```

Este algoritmo se ejecuta en una computadora que procesa 1.000 operaciones por segundo. Determinar el tiempo aproximado que requerirá el algoritmo para resolver un problema de tamaño $n = 4.096$.

Iteraciones del primer for:

$i = 1.$
 $i = 2.$
 $i = 4.$
 $\dots$
 $i = 2^{k-1}.$

$2^{k-1} = n$
 $(k - 1) \log_2 2 = \log_2 n$
 $(k - 1) * 1 = \log_2 n$
 $k - 1 = \log_2 n$
 $k = \log_2 n + 1.$

Iteraciones del segundo for:

$j = 0.$
 $j = \frac{n}{2}.$

Entonces:

$T(n) = cte_1 + \sum_{i=1}^{\log_2 n + 1} \sum_{j=1}^2 cte_2$
 $T(n) = cte_1 + \sum_{i=1}^{\log_2 n + 1} 2cte_2$
 $T(n) = cte_1 + (\log_2 n + 1) * 2cte_2$
 $T(n) = cte_1 + 2 (\log_2 n + 1) cte_2.$

$T(4096) \cong \log_2 4096$
 $T(4096) \cong 12.$

1.000 operaciones por segundo.

$$\text{Tiempo (en segundos)} \cong \frac{12}{1000}$$

$$\text{Tiempo (en segundos)} \cong 0,012.$$

Por lo tanto, el tiempo aproximado que requerirá el algoritmo para resolver un problema de tamaño $n=4.096$ es 0,012 segundos.